SLIP 1

Q1

```java
class AlphabetThread extends Thread
{
    public void run()
    {
        try
        {
            for(char ch = 'A'; ch <= 'Z'; ch++)
            {
                System.out.println(ch);
                Thread.sleep(2000); // 2 seconds delay
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

class Slip1Q1
{
    public static void main(String args[])
    {
        AlphabetThread t = new AlphabetThread();

        t.setPriority(8); // Set thread priority

        t.start();
    }
}
```

Q2

```sql
CREATE TABLE Employee
(
    Eno INT PRIMARY KEY,
    EName VARCHAR(50),
    Designation VARCHAR(50),
```

```
    Salary FLOAT
);


import java.sql.*;
import java.util.Scanner;

class EmployeeDB
{
    Connection con;
    Scanner sc = new Scanner(System.in);

    EmployeeDB()
    {
        try
        {
            Class.forName("org.postgresql.Driver");

            con = DriverManager.getConnection(
                "jdbc:postgresql://192.168.5.200:5432/bcaty12",
                "bcaty12",
                ""
            );

            System.out.println("Connected to PostgreSQL");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    void insertEmployee()
    {
        try
        {
            System.out.print("Enter Employee No: ");
            int eno = sc.nextInt();
            sc.nextLine();

            System.out.print("Enter Employee Name: ");
            String name = sc.nextLine();
```

```
            System.out.print("Enter Designation: ");
            String desig = sc.nextLine();

            System.out.print("Enter Salary: ");
            float sal = sc.nextFloat();
            sc.nextLine();

            PreparedStatement ps = con.prepareStatement(
                "INSERT INTO Employee VALUES (?, ?, ?, ?)"
            );

            ps.setInt(1, eno);
            ps.setString(2, name);
            ps.setString(3, desig);
            ps.setFloat(4, sal);

            ps.executeUpdate();
            System.out.println("Record inserted.");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    void viewAll()
    {
        try
        {
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM Employee");

            System.out.println("\n--- Employee Records ---");

            while(rs.next())
            {
                System.out.println(
                    rs.getInt(1) + " | " +
                    rs.getString(2) + " | " +
                    rs.getString(3) + " | " +
                    rs.getFloat(4)
                );
            }
```

```
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

class MainClass
{
    public static void main(String args[])
    {
        EmployeeDB obj = new EmployeeDB();
        Scanner sc = new Scanner(System.in);

        int choice;

        do
        {
            System.out.println("\n--- MENU ---");
            System.out.println("1. Insert Employee");
            System.out.println("2. View All Employees");
            System.out.println("3. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch(choice)
            {
                case 1: obj.insertEmployee(); break;
                case 2: obj.viewAll(); break;
                case 3: System.out.println("Program Ended"); break;
                default: System.out.println("Invalid choice");
            }

        } while(choice != 3);
    }
}
```

SLIP 2

Q1

```java
import java.util.*;

class Slip2Q1
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of friends: ");
        int n = sc.nextInt();
        sc.nextLine();

        HashSet<String> hs = new HashSet<String>();

        for(int i = 1; i <= n; i++)
        {
            System.out.print("Enter name " + i + ": ");
            String name = sc.nextLine();
            hs.add(name);
        }

        // Convert to TreeSet for sorting
        TreeSet<String> ts = new TreeSet<String>(hs);

        System.out.println("\nNames in Ascending Order:");

        Iterator<String> itr = ts.iterator();

        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }
    }
}
```

Q2

```html
<html>
<head>
    <title>Login Page</title>
</head>

<body>

<h2>Login Form</h2>

<form method="post" action="LoginServlet">

Username:
<input type="text" name="uname"><br><br>

Password:
<input type="password" name="pass"><br><br>

<input type="submit" value="Login">

</form>

</body>
</html>
```

```java
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet
{
    public void doPost(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException
    {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        String uname = req.getParameter("uname");
        String pass = req.getParameter("pass");

        if(uname.equals("admin") && pass.equals("admin"))
```

```
        {
            out.println("<h1>Welcome " + uname + "</h1>"); //Edhar you
can use sendRedirect() function, but you have to make 2 html for that,
example: res.sendRedirect("Welcome.html");
        }
        else
        {
            out.println("<h1>Invalid Username or Password</h1>");
        }
    }
}
```

```
<web-app>

<servlet>
    <servlet-name>LoginServlet</servlet-name>
    <servlet-class>LoginServlet</servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name>LoginServlet</servlet-name>
    <url-pattern>/LoginServlet</url-pattern>
</servlet-mapping>

</web-app>
```

SLIP 3

Q1

```
import java.util.*;

class Slip3Q1
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        LinkedList<String> list = new LinkedList<String>();

        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        sc.nextLine();
```

```java
        // Add elements at end
        for(int i = 1; i <= n; i++)
        {
            System.out.print("Enter element " + i + ": ");
            String str = sc.nextLine();
            list.add(str);   // adds at end
        }

        // Display in reverse order
        System.out.println("\nElements in Reverse Order:");

        ListIterator<String> itr = list.listIterator(list.size());

        while(itr.hasPrevious())
        {
            System.out.println(itr.previous());
        }
    }
}
```

Q2

```sql
CREATE TABLE Patient
(
    PNo INT PRIMARY KEY,
    PName VARCHAR(50),
    Age INT,
    Disease VARCHAR(50)
);

INSERT INTO Patient VALUES (1, 'Rahul', 25, 'Fever');
INSERT INTO Patient VALUES (2, 'Sneha', 30, 'Diabetes');
```

```html
<%@ page import="java.sql.*" %>

<html>
<head>
    <title>Patient Details</title>
</head>

<body>
```

```
<h2>Patient Records</h2>

<table border="1">
<tr>
    <th>PNo</th>
    <th>PName</th>
    <th>Age</th>
    <th>Disease</th>
</tr>

<%
try
{
    Class.forName("org.postgresql.Driver");

    Connection con = DriverManager.getConnection(
        "jdbc:postgresql://localhost:5432/postgres",
        "postgres",
        "password"
    );

    Statement st = con.createStatement();

    ResultSet rs = st.executeQuery("SELECT * FROM Patient");

    while(rs.next())
    {
%>

<tr>
    <td><%= rs.getInt(1) %></td>
    <td><%= rs.getString(2) %></td>
    <td><%= rs.getInt(3) %></td>
    <td><%= rs.getString(4) %></td>
</tr>

<%
    }

    con.close();
}
catch(Exception e)
```

```
{
    out.println(e);
}
%>

</table>

</body>
</html>
```

SLIP 4

Q1

```
import java.util.*;

class Slip4Q1
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        // Using HashMap (city → STD code)
        HashMap<String, Integer> map = new HashMap<String, Integer>();

        int choice;

        do
        {
            System.out.println("\n--- MENU ---");
            System.out.println("1. Add City");
            System.out.println("2. Search City");
            System.out.println("3. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch(choice)
            {
                case 1:
                    System.out.print("Enter city name: ");
                    String city = sc.nextLine();
```

```
                    if(map.containsKey(city))
                    {
                        System.out.println("City already exists (No
duplicates allowed)");
                    }
                    else
                    {
                        System.out.print("Enter STD code: ");
                        int code = sc.nextInt();
                        sc.nextLine();

                        map.put(city, code);
                        System.out.println("City added.");
                    }
                    break;

                case 2:
                    System.out.print("Enter city to search: ");
                    String searchCity = sc.nextLine();

                    if(map.containsKey(searchCity))
                    {
                        System.out.println("STD Code: " +
map.get(searchCity));
                    }
                    else
                    {
                        System.out.println("City not found.");
                    }
                    break;

                case 3:
                    System.out.println("Program Ended");
                    break;

                default:
                    System.out.println("Invalid choice");
            }

        } while(choice != 3);
    }
}
```

Q2

```
import java.awt.*;
import javax.swing.*;

class BlinkText extends JFrame implements Runnable
{
    JLabel lbl;
    Thread t;

    BlinkText()
    {
        setTitle("Blinking Text");

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        lbl = new JLabel("BLINKING TEXT", SwingConstants.CENTER);
        lbl.setBounds(80, 80, 200, 30);
SL
        add(lbl);

        setLayout(null);
        setSize(300, 200);
        setVisible(true);

        // Create thread using Runnable
        t = new Thread(this);
        t.start();
    }

    public void run()
    {
        try
        {
            while(true)
            {
                lbl.setVisible(false);
                Thread.sleep(500);

                lbl.setVisible(true);
                Thread.sleep(500);
            }
        }
        catch(Exception e)
```

```
        {
            System.out.println(e);
        }
    }

    public static void main(String args[])
    {
        new BlinkText();
    }
}
```

SLIP 5

Q1

```
import java.util.*;

class Slip5Q1
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        // TreeSet → Sorted + No duplicates
        TreeSet<Integer> ts = new TreeSet<Integer>();

        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();

        // Accept elements
        for(int i = 1; i <= n; i++)
        {
            System.out.print("Enter number " + i + ": ");
            int num = sc.nextInt();
            ts.add(num);
        }

        // Display using Iterator
        System.out.println("\nSorted Elements:");
        Iterator<Integer> itr = ts.iterator();
```

```
        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }

        // Search element
        System.out.print("\nEnter element to search: ");
        int key = sc.nextInt();

        if(ts.contains(key))
        {
            System.out.println("Element found.");
        }
        else
        {
            System.out.println("Element not found.");
        }
    }
}
```

Q2

```
import java.util.Random;


// Thread 1: Generate random number every 1 second
class NumberGenerator extends Thread {
    public void run() {
        Random r = new Random();


        while (true)
      //for(int i = 1; i <= 10; i++)
       {
            int num = r.nextInt(100); // generate number (0-99)
            System.out.println("\nGenerated Number: " + num);


            if (num % 2 == 0) {
                SquareThread t2 = new SquareThread(num);
                t2.start();
```

```java
            } else {
                CubeThread t3 = new CubeThread(num);
                t3.start();
            }


            try {
                Thread.sleep(1000); // wait for 1 second
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}


// Thread 2: Compute square
class SquareThread extends Thread {
    int num;


    SquareThread(int num) {
        this.num = num;
    }


    public void run() {
        System.out.println("Square of " + num + " = " + (num * num));
    }
}


// Thread 3: Compute cube
class CubeThread extends Thread {
    int num;


    CubeThread(int num) {
        this.num = num;
    }


    public void run() {
```

```
        System.out.println("Cube of " + num + " = " + (num * num *
num));
    }
}


// Main class
class A1{
    public static void main(String args[]) {
        NumberGenerator t1 = new NumberGenerator();
        t1.start();
    }
}
```

SLIP 6

Q1

```
class ReverseThread extends Thread
{
    public void run()
    {
        try
        {
            for(int i = 100; i >= 1; i--)
            {
                System.out.println(i);
                Thread.sleep(2000); // sleep for 2 seconds
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

class Slip6Q1
{
```

```
    public static void main(String args[])
    {
        ReverseThread t = new ReverseThread();

        // Change thread name
        t.setName("reverse order");

        // Display thread name
        System.out.println("Thread Name: " + t.getName());

        t.start();
    }
}
```

Q2

```
CREATE TABLE Product
(
    Pid INT PRIMARY KEY,
    Pname VARCHAR(50),
    Price FLOAT
);


import java.sql.*;
import java.util.Scanner;

class ProductDB
{
    Connection con;
    Scanner sc = new Scanner(System.in);

    ProductDB()
    {
        try
        {
            Class.forName("org.postgresql.Driver");

            con = DriverManager.getConnection(
                "jdbc:postgresql://192.168.5.200:5432/bcaty12",
                "bcaty12",
```

```
                "
            );

            System.out.println("Connected to PostgreSQL");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    void insertProduct()
    {
        try
        {
            System.out.print("Enter Product ID: ");
            int pid = sc.nextInt();
            sc.nextLine();

            System.out.print("Enter Product Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Price: ");
            float price = sc.nextFloat();
            sc.nextLine();

            PreparedStatement ps = con.prepareStatement(
                "INSERT INTO Product VALUES (?, ?, ?)"
            );

            ps.setInt(1, pid);
            ps.setString(2, name);
            ps.setFloat(3, price);

            ps.executeUpdate();
            System.out.println("Record inserted.");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
```

```java
void viewAll()
{
    try
    {
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("SELECT * FROM Product");

        System.out.println("\n--- Product Records ---");

        while(rs.next())
        {
            System.out.println(
                rs.getInt(1) + " | " +
                rs.getString(2) + " | " +
                rs.getFloat(3)
            );
        }
    }
    catch(Exception e)
    {
        System.out.println(e);
    }
}

void viewPriceGreater()
{
    try
    {
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery(
            "SELECT * FROM Product WHERE Price > 1000"
        );

        System.out.println("\n--- Products with Price > 1000 ---");

        while(rs.next())
        {
            System.out.println(
                rs.getInt(1) + " | " +
                rs.getString(2) + " | " +
                rs.getFloat(3)
            );
        }
```

```
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

class MainClass
{
    public static void main(String args[])
    {
        ProductDB obj = new ProductDB();
        Scanner sc = new Scanner(System.in);

        int choice;

        do
        {
            System.out.println("\n--- MENU ---");
            System.out.println("1. Insert Product");
            System.out.println("2. View All Products");
            System.out.println("3. View Products Price > 1000");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch(choice)
            {
                case 1: obj.insertProduct(); break;
                case 2: obj.viewAll(); break;
                case 3: obj.viewPriceGreater(); break;
                case 4: System.out.println("Program Ended"); break;
                default: System.out.println("Invalid choice");
            }

        } while(choice != 4);
    }
}
```

SLIP 7

Q1

```java
class PrintThread extends Thread
{
    String msg;
    int n;

    // Constructor
    PrintThread(String msg, int n)
    {
        this.msg = msg;
        this.n = n;
    }

    public void run()
    {
        for(int i = 1; i <= n; i++)
        {
            System.out.println(msg + " : " + i);
        }
    }
}

class Slip7Q1
{
    public static void main(String args[])
    {
        // Create 3 threads with given values
        PrintThread t1 = new PrintThread("COVID19", 10);
        PrintThread t2 = new PrintThread("LOCKDOWN2020", 20);
        PrintThread t3 = new PrintThread("VACCINATED2021", 30);

        // Start threads
        t1.start();
        t2.start();
        t3.start();
    }
}
```

Q2

```html
<html>
<head>
<title>Arithmetic Operations</title>
</head>

<body>

<h2>Arithmetic Operations</h2>

<form action="result.jsp" method="post">

Enter Number 1:
<input type="text" name="n1"><br><br>

Enter Number 2:
<input type="text" name="n2"><br><br>

Select Operation:<br>
<input type="radio" name="op" value="add"> Addition<br>
<input type="radio" name="op" value="sub"> Subtraction<br>
<input type="radio" name="op" value="mul"> Multiplication<br>
<input type="radio" name="op" value="div"> Division<br><br>

<input type="submit" value="Calculate">

</form>

</body>
</html>
```

```jsp
<%@ page language="java" %>

<html>
<body>

<%
    int n1 = Integer.parseInt(request.getParameter("n1"));
    int n2 = Integer.parseInt(request.getParameter("n2"));
    String op = request.getParameter("op");

    int result = 0;
```

```
    if(op.equals("add"))
        result = n1 + n2;

    else if(op.equals("sub"))
        result = n1 - n2;

    else if(op.equals("mul"))
        result = n1 * n2;

    else if(op.equals("div"))
        result = n1 / n2;
%>

<h2>Result = <%= result %></h2>

</body>
</html>
```

SLIP 8

Q1

```
import java.util.*;

class Slip8Q1
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        LinkedList<String> list = new LinkedList<String>();

        System.out.print("Enter number of friends: ");
        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        // Accept names
        for(int i = 1; i <= n; i++)
        {
            System.out.print("Enter name " + i + ": ");
            String name = sc.nextLine();
```

```
            list.add(name);
        }

        // Display using ListIterator
        System.out.println("\nFriend Names:");

        ListIterator<String> itr = list.listIterator();

        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }

        sc.close();
    }
}
```

Q2

```
CREATE TABLE DONAR
(
    Dno INT,
    Dname VARCHAR(20),
    City VARCHAR(20)
);


import java.sql.*;

class DonarMetaData
{
    Connection con;

    DonarMetaData()
    {
        try
        {
            Class.forName("org.postgresql.Driver");

            con = DriverManager.getConnection(
                "jdbc:postgresql://192.168.5.200:5432/bcaty12",
```

```
                "bcaty12",
                ""
            );

            System.out.println("Connected to PostgreSQL");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    void showMetaData()
    {
        try
        {
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM DONAR");

            ResultSetMetaData rsmd = rs.getMetaData();

            int count = rsmd.getColumnCount();

            System.out.println("\n--- DONAR Table Column Information
---");

            for(int i = 1; i <= count; i++)
            {
                System.out.println("Column No: " + i);
                System.out.println("Column Name: " +
rsmd.getColumnName(i));
                System.out.println("Column Type: " +
rsmd.getColumnTypeName(i));
                System.out.println("Column Size: " +
rsmd.getColumnDisplaySize(i));
                System.out.println("----------------------------");
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
```

```
}

class MainClass
{
    public static void main(String args[])
    {
        DonarMetaData obj = new DonarMetaData();
        obj.showMetaData();
    }
}



OR


import java.sql.*;
import java.util.Scanner;

class DonarDB
{
    Connection con;
    Scanner sc = new Scanner(System.in);

    DonarDB()
    {
        try
        {
            Class.forName("org.postgresql.Driver");

            con = DriverManager.getConnection(
                "jdbc:postgresql://192.168.5.200:5432/bcaty12",
                "bcaty12",
                ""
            );

            System.out.println("Connected to PostgreSQL");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
```

```java
// 🔹 Insert Record
void insertDonar()
{
    try
    {
        System.out.print("Enter Donar ID: ");
        int id = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Donar Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Blood Group: ");
        String bg = sc.nextLine();

        PreparedStatement ps = con.prepareStatement(
            "INSERT INTO DONAR VALUES (?, ?, ?)"
        );

        ps.setInt(1, id);
        ps.setString(2, name);
        ps.setString(3, bg);

        ps.executeUpdate();
        System.out.println("Record inserted.");
    }
    catch(Exception e)
    {
        System.out.println(e);
    }
}

// 🔹 Display Column Info using ResultSetMetaData
void showMetaData()
{
    try
    {
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("SELECT * FROM DONAR");

        ResultSetMetaData rsmd = rs.getMetaData();
```

```
            int count = rsmd.getColumnCount();

            System.out.println("\n--- DONAR Table Column Information
---");

            for(int i = 1; i <= count; i++)
            {
                System.out.println("Column No: " + i);
                System.out.println("Column Name: " +
rsmd.getColumnName(i));
                System.out.println("Column Type: " +
rsmd.getColumnTypeName(i));
                System.out.println("Column Size: " +
rsmd.getColumnDisplaySize(i));
                System.out.println("----------------------------");
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

class MainClass
{
    public static void main(String args[])
    {
        DonarDB obj = new DonarDB();
        Scanner sc = new Scanner(System.in);

        int choice;

        do
        {
            System.out.println("\n--- MENU ---");
            System.out.println("1. Insert Donar");
            System.out.println("2. Show Column Info");
            System.out.println("3. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine();
```

```
            switch(choice)
            {
                case 1: obj.insertDonar(); break;
                case 2: obj.showMetaData(); break;
                case 3: System.out.println("Program Ended"); break;
                default: System.out.println("Invalid choice");
            }

        } while(choice != 3);
    }
}
```

SLIP 9

Q1

```
<html>
<body>

<h2>Check Perfect Number</h2>

<form action="perfect.jsp" method="post">
    Enter Number:
    <input type="text" name="num"><br><br>

    <input type="submit" value="Check">
</form>

</body>
</html>

<%@ page language="java" %>

<html>
<body>

<%
    int n = Integer.parseInt(request.getParameter("num"));
    int sum = 0;

    for(int i = 1; i <= n/2; i++)
    {
```

```
        if(n % i == 0)
        {
            sum += i;
        }
    }

    if(sum == n)
    {
%>
        <h2><%= n %> is a Perfect Number</h2>
<%
    }
    else
    {
%>
        <h2><%= n %> is NOT a Perfect Number</h2>
<%
    }
%>

</body>
</html>
```

Q2

```
CREATE TABLE Employee
(
    Id INT,
    name VARCHAR(20),
    salary INT
);
```

```
import java.sql.*;
import java.util.Scanner;

class EmployeeDB
{
    Connection con;
    Scanner sc = new Scanner(System.in);

    EmployeeDB()
    {
```

```
        try
        {
            Class.forName("org.postgresql.Driver");

            con = DriverManager.getConnection(
                "jdbc:postgresql://192.168.5.200:5432/bcaty12",
                "bcaty12",
                ""
            );

            System.out.println("Connected to PostgreSQL");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    // 🔹 Insert Employee using PreparedStatement
    void insertEmployee()
    {
        try
        {
            System.out.print("Enter Employee ID: ");
            int id = sc.nextInt();
            sc.nextLine();

            System.out.print("Enter Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Salary: ");
            float salary = sc.nextFloat();
            sc.nextLine();

            PreparedStatement ps = con.prepareStatement(
                "INSERT INTO Employee VALUES (?, ?, ?)"
            );

            ps.setInt(1, id);
            ps.setString(2, name);
            ps.setFloat(3, salary);

            ps.executeUpdate();
```

```
            System.out.println("Record Inserted.");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    // 🔹 Display employees whose name starts with 'A'
    void displayAEmployees()
    {
        try
        {
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery(
                "SELECT * FROM Employee WHERE name LIKE 'A%'"
            );

            System.out.println("\n--- Employees with Name starting with
A ---");

            while(rs.next())
            {
                System.out.println(
                    rs.getInt(1) + " | " +
                    rs.getString(2) + " | " +
                    rs.getFloat(3)
                );
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

class MainClass
{
    public static void main(String args[])
    {
        EmployeeDB obj = new EmployeeDB();
        Scanner sc = new Scanner(System.in);
```

```
        int choice;

        do
        {
            System.out.println("\n--- MENU ---");
            System.out.println("1. Insert Employee");
            System.out.println("2. Display Names starting with A");
            System.out.println("3. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch(choice)
            {
                case 1: obj.insertEmployee(); break;
                case 2: obj.displayAEmployees(); break;
                case 3: System.out.println("Program Ended"); break;
                default: System.out.println("Invalid choice");
            }

        } while(choice != 3);
    }
}
```

SLIP 10

Q1

```
import java.util.*;

class Slip4Q1
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        // Using HashMap (city → STD code)
        HashMap<String, Integer> map = new HashMap<String, Integer>();

        int choice;
```

```
        do
        {
            System.out.println("\n--- MENU ---");
            System.out.println("1. Add City");
            System.out.println("2. Search City");
            System.out.println("3. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch(choice)
            {
                case 1:
                    System.out.print("Enter city name: ");
                    String city = sc.nextLine();

                    if(map.containsKey(city))
                    {
                        System.out.println("City already exists (No
duplicates allowed)");
                    }
                    else
                    {
                        System.out.print("Enter STD code: ");
                        int code = sc.nextInt();
                        sc.nextLine();

                        map.put(city, code);
                        System.out.println("City added.");
                    }
                    break;

                case 2:
                    System.out.print("Enter city to search: ");
                    String searchCity = sc.nextLine();

                    if(map.containsKey(searchCity))
                    {
                        System.out.println("STD Code: " +
map.get(searchCity));
                    }
                    else
                    {
```

```
                        System.out.println("City not found.");
                    }
                    break;

                case 3:
                    System.out.println("Program Ended");
                    break;

                default:
                    System.out.println("Invalid choice");
            }

        } while(choice != 3);
    }
}
```

Q2

```
import java.sql.*;

class Slip10Q2
{
    public static void main(String args[])
    {
        try
        {
            // Load Driver
            Class.forName("org.postgresql.Driver");

            // Connection
            Connection con = DriverManager.getConnection(
                "jdbc:postgresql://localhost:5432/postgres",
                "postgres",
                "password"
            );

            // Get DatabaseMetaData
            DatabaseMetaData dbmd = con.getMetaData();

            // Display database information
            System.out.println("Driver Name: " + dbmd.getDriverName());
```

```
            System.out.println("Driver Version: " +
dbmd.getDriverVersion());
            System.out.println("Database Product Name: " +
dbmd.getDatabaseProductName());
            System.out.println("Database Version: " +
dbmd.getDatabaseProductVersion());
            System.out.println("User Name: " + dbmd.getUserName());

            // Get all tables
            System.out.println("\nTables in the Database:");

            ResultSet rs = dbmd.getTables(null, null, "%", new String[]
{"TABLE"});

            while(rs.next())
            {
                System.out.println(rs.getString("TABLE_NAME"));
            }

            con.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

SLIP 11

Q1

```
<html>
<body>

<h2>Sum of Digits</h2>

<form action="sum.jsp" method="post">
    Enter Number:
    <input type="text" name="num"><br><br>
```

```
    <input type="submit" value="Calculate">
</form>

</body>
</html>

<%@ page language="java" %>

<html>
<body>

<%
    int n = Integer.parseInt(request.getParameter("num"));
    int sum = 0;

    while(n > 0)
    {
        int digit = n % 10;
        sum += digit;
        n = n / 10;
    }
%>

<h2 style="color:red;">
    Sum of Digits = <%= sum %>
</h2>

</body>
</html>
```

Q2

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class NumberThread extends Thread
{
    JTextField tf;
```

```java
    NumberThread(JTextField tf)
    {
        this.tf = tf;
    }

    public void run()
    {
        try
        {
            for(int i = 1; i <= 100; i++)
            {
                tf.setText(String.valueOf(i));
                Thread.sleep(100); // delay for smooth display
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

class Slip11Q2 extends JFrame implements ActionListener
{
    JTextField tf;
    JButton btn;

    Slip11Q2()
    {
        // Close operation (IMPORTANT)
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        tf = new JTextField();
        tf.setBounds(100, 50, 150, 30);

        btn = new JButton("Start");
        btn.setBounds(100, 100, 100, 30);

        btn.addActionListener(this);

        add(tf);
        add(btn);
```

```
        setLayout(null);
        setSize(350, 250);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e)
    {
        NumberThread t = new NumberThread(tf);
        t.start(); // start thread on button click
    }

    public static void main(String args[])
    {
        new Slip11Q2();
    }
}
```

SLIP 12

Q1

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("Thread Name: " + getName());
        System.out.println("Thread Priority: " + getPriority());
    }
}

class Slip12Q1
{
    public static void main(String args[])
    {
        MyThread t = new MyThread();

        // Set thread name
        t.setName("MyThread");

        // Set priority to 7
```

```
        t.setPriority(7);

        t.start();
    }
}
```

OR(Unnati's)

```
class ThreadInfo extends Thread
{
    public void run()
    {
        // Display current thread name and priority
        System.out.println("Thread Name: " +
Thread.currentThread().getName());
        System.out.println("Thread Priority: " +
Thread.currentThread().getPriority());

        // Change priority to 7
        Thread.currentThread().setPriority(7);

        // Display updated priority
        System.out.println("Updated Priority: " +
Thread.currentThread().getPriority());
    }
    public static void main(String[] args)
    {
        ThreadInfo t1 = new ThreadInfo();
        t1.setName("MyThread"); // Set thread name
        t1.start(); // Start thread
    }
}
```

Q2

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class VisitServlet extends HttpServlet
{
```

```java
    public void doGet(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException
    {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        int count = 0;

        // Get cookies
        Cookie cookies[] = req.getCookies();

        boolean found = false;

        if(cookies != null)
        {
            for(int i = 0; i < cookies.length; i++)
            {
                if(cookies[i].getName().equals("visit"))
                {
                    found = true;
                    count = Integer.parseInt(cookies[i].getValue());
                    count++;

                    out.println("<h2>You visited " + count + "
times</h2>");

                    // Update cookie
                    cookies[i].setValue(String.valueOf(count));
                    res.addCookie(cookies[i]);
                    break;
                }
            }
        }

        // First visit
        if(!found)
        {
            count = 1;
            out.println("<h2>Welcome! This is your first visit</h2>");

            Cookie c = new Cookie("visit", String.valueOf(count));
            res.addCookie(c);
        }
```

```
    }
}

<servlet>
    <servlet-name>VisitServlet</servlet-name>
    <servlet-class>VisitServlet</servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name>VisitServlet</servlet-name>
    <url-pattern>/VisitServlet</url-pattern>
</servlet-mapping>
```

SLIP 13

Q1

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("Thread Name: " + getName());
        System.out.println("Thread Priority: " + getPriority());

        // Check if thread is alive
        System.out.println("Is Thread Alive? " + isAlive());
    }
}

class Slip13Q1
{
    public static void main(String args[])
    {
        MyThread t = new MyThread();

        // Change thread name
        t.setName("Java Thread");

        // Set priority (default is 5, but we can show it)
        t.setPriority(7);
```

```
        // Start thread
        t.start();

        // Check alive from main also
        System.out.println("Alive from main: " + t.isAlive());
    }
}
```

Q2

```
import java.util.*;

class Slip13Q2
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        // Main LinkedList
        LinkedList<String> list = new LinkedList<String>();

        System.out.print("Enter number of colors: ");
        int n = sc.nextInt();
        sc.nextLine();

        // Accept colors from user
        for(int i = 1; i <= n; i++)
        {
            System.out.print("Enter color " + i + ": ");
            list.add(sc.nextLine());
        }

        int choice;

        do
        {
            System.out.println("\n1. Display using Iterator");
            System.out.println("2. Display in Reverse using
ListIterator");
            System.out.println("3. Insert new colors between two
colors");
```

```
            System.out.println("4. Exit");

            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch(choice)
            {
                case 1:
                    // Iterator display
                    Iterator<String> itr = list.iterator();
                    System.out.println("List:");
                    while(itr.hasNext())
                    {
                        System.out.println(itr.next());
                    }
                    break;

                case 2:
                    // Reverse using ListIterator
                    ListIterator<String> litr =
list.listIterator(list.size());
                    System.out.println("Reverse List:");
                    while(litr.hasPrevious())
                    {
                        System.out.println(litr.previous());
                    }
                    break;

                case 3:
                    // Second list input
                    LinkedList<String> list2 = new
LinkedList<String>();

                    System.out.print("Enter number of colors to insert:
");
                    int m = sc.nextInt();
                    sc.nextLine();

                    for(int i = 1; i <= m; i++)
                    {
                        System.out.print("Enter color: ");
                        list2.add(sc.nextLine());
```

```
                    }

                    // Position input
                    System.out.print("Enter color after which to
insert: ");
                    String after = sc.nextLine();

                    System.out.print("Enter color before which to
insert: ");
                    String before = sc.nextLine();

                    int pos = list.indexOf(after);

                    if(pos != -1 && list.contains(before))
                    {
                        // Insert at correct position
                        list.addAll(pos + 1, list2);
                        System.out.println("Elements inserted
successfully!");
                    }
                    else
                    {
                        System.out.println("Invalid position!");
                    }
                    break;

                case 4:
                    System.out.println("Exiting...");
                    break;

                default:
                    System.out.println("Invalid choice!");
            }

        } while(choice != 4);

        sc.close();
    }
}
```

SLIP 14

Q1

```java
import java.util.*;

class Slip14Q1
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        // HashMap (BookName → Price)
        HashMap<String, Float> hm = new HashMap<String, Float>();

        System.out.print("Enter number of books: ");
        int n = sc.nextInt();
        sc.nextLine();

        // Accept input
        for(int i = 1; i <= n; i++)
        {
            System.out.print("Enter book name: ");
            String name = sc.nextLine();

            System.out.print("Enter price: ");
            float price = sc.nextFloat();
            sc.nextLine();

            hm.put(name, price);
        }

        // Display using entrySet
        System.out.println("\n--- Book Details ---");

        for(Map.Entry<String, Float> e : hm.entrySet())
        {
            System.out.println(
                "Book: " + e.getKey() +
                " | Price: " + e.getValue()
            );
        }
```

```
        sc.close();
    }
}
```

Q2

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.Date;

public class InfoServlet extends HttpServlet
{
    public void doGet(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException
    {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        // 1. Get IP Address
        String ip = req.getRemoteAddr();

        // 2. Get Browser Info (User-Agent)
        String browser = req.getHeader("User-Agent");

        // 3. Get Server Date & Time
        Date d = new Date();

        // Display Output
        out.println("<html><body>");
        out.println("<h2>Client IP Address: " + ip + "</h2>");
        out.println("<h2>Browser Type: " + browser + "</h2>");
        out.println("<h2>Server Date & Time: " + d + "</h2>");
        out.println("</body></html>");
    }
}

<servlet>
    <servlet-name>InfoServlet</servlet-name>
    <servlet-class>InfoServlet</servlet-class>
</servlet>

<servlet-mapping>
```

```
    <servlet-name>InfoServlet</servlet-name>
    <url-pattern>/InfoServlet</url-pattern>
</servlet-mapping>
```

SLIP 15

Q1

```
<html>
<body>

<h2>Enter a Number</h2>

<form action="number.jsp">
    Enter Number:
    <input type="text" name="num">
    <br><br>
    <input type="submit" value="Convert">
</form>

</body>
</html>

<%@ page import="java.util.*" %>

<html>
<body>

<%
    String num = request.getParameter("num");

    String words[] = {
        "Zero","One","Two","Three","Four",
        "Five","Six","Seven","Eight","Nine"
    };

    out.println("<h2 style='color:red;'>");

    for(int i = 0; i < num.length(); i++)
    {
        char ch = num.charAt(i);

        if(Character.isDigit(ch))
```

```
        {
            int digit = ch - '0';
            out.print(words[digit] + " ");
        }
    }

    out.println("</h2>");
%>

</body>
</html>
```

Q2

```
import javax.swing.*;
import java.awt.event.*;

// Thread class
class CharThread extends Thread
{
    JTextField tf;

    CharThread(JTextField tf)
    {
        this.tf = tf;
    }

    public void run()
    {
        try
        {
            for(char ch = 'A'; ch <= 'Z'; ch++)
            {
                tf.setText(tf.getText() + ch);
                Thread.sleep(200); // delay for visibility
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
```

```java
    }
}

// Main Frame
class Slip15Q2 extends JFrame implements ActionListener
{
    JTextField tf;
    JButton btn;

    Slip15Q2()
    {
        // Close operation
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        tf = new JTextField();
        tf.setBounds(50, 50, 200, 30);

        btn = new JButton("Start");
        btn.setBounds(80, 100, 100, 30);

        btn.addActionListener(this);

        add(tf);
        add(btn);

        setLayout(null);
        setSize(300, 200);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e)
    {
        // Start thread on button click
        CharThread t = new CharThread(tf);
        t.start();
    }

    public static void main(String args[])
    {
        new Slip15Q2();
    }
}
```

SLIP 16

Q1. Write a program in to display A to Z characters after every 2 seconds. Create a thread and display information of thread as name of thread, priority of the thread. [8 Marks]

```
class MyThread extends Thread
{
    public void run()
    {
        try
        {
            // Display thread info
            System.out.println("Thread Name: " + getName());
            System.out.println("Thread Priority: " + getPriority());

            // Print A to Z with 2 sec delay
            for(char ch = 'A'; ch <= 'Z'; ch++)
            {
                System.out.println(ch);
                Thread.sleep(2000); // 2 seconds
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

class Slip16Q1
{
    public static void main(String args[])
    {
        MyThread t = new MyThread();

        // Set name and priority
        t.setName("Alphabet Thread");
        t.setPriority(7);

        t.start();
```

```
    }
}
```

Q2. Accept username and password from HTML. Write a JSP program to search them into database, if found then display appropriate message on the browser otherwise display error message. [14 Marks]

```
CREATE TABLE UserLogin
(
    username VARCHAR(20),
    password VARCHAR(20)
);

INSERT INTO UserLogin VALUES('admin','123');

login.html
<html>
<body>

<h2>Login Form</h2>

<form action="login.jsp" method="post">
    Username:
    <input type="text" name="uname"><br><br>

    Password:
    <input type="password" name="pass"><br><br>

    <input type="submit" value="Login">
</form>

</body>
</html>


login.jsp

<%@ page import="java.sql.*" %>

<html>
<body>
```

```
<%
String uname = request.getParameter("uname");
String pass = request.getParameter("pass");

try
{
    Class.forName("org.postgresql.Driver");

    Connection con = DriverManager.getConnection(
        "jdbc:postgresql://localhost:5432/postgres",
        "postgres",
        "password"
    );

    PreparedStatement ps = con.prepareStatement(
        "SELECT * FROM UserLogin WHERE username=? AND password=?"
    );

    ps.setString(1, uname);
    ps.setString(2, pass);

    ResultSet rs = ps.executeQuery();

    if(rs.next())
    {
%>
        <h2 style="color:green;">Login Successful!</h2>
<%
    }
    else
    {
%>
        <h2 style="color:red;">Invalid Username or Password</h2>
<%
    }

    con.close();
}
catch(Exception e)
{
    out.println(e);
}
%>
```

```
</body>
</html>
```

SLIP 17

Q1. Write a java program to accept names of 'n' cities, insert same into ArrayList collection and display the contents of same array list. Remove the element of ArrayList at particular index. Accepting index from user. [8 Marks]

```java
import java.util.*;

class Slip17Q1
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        ArrayList<String> list = new ArrayList<String>();

        System.out.print("Enter number of cities: ");
        int n = sc.nextInt();
        sc.nextLine();

        // Accept cities
        for(int i = 1; i <= n; i++)
        {
            System.out.print("Enter city " + i + ": ");
            list.add(sc.nextLine());
        }

        // Display using Iterator
        System.out.println("\n--- City List ---");
        Iterator<String> itr = list.iterator();

        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }

        // Remove element at index
        System.out.print("\nEnter index to remove: ");
```

```java
        int index = sc.nextInt();

        if(index >= 0 && index < list.size())
        {
            list.remove(index);
            System.out.println("Element removed successfully.");
        }
        else
        {
            System.out.println("Invalid index!");
        }

        // Display updated list
        System.out.println("\n--- Updated List ---");
        itr = list.iterator();

        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }

        sc.close();
    }
}
```

Q2. Write a java program to solve producer consumer problem in which a producer produces a value and consumer consume the value before producer generate the next value. (Use thread synchronization) [1 4 Marks]

```java
class Shared
{
    int data;
    boolean hasValue = false;

    // Produce value
    synchronized void produce(int value)
    {
        try
        {
            while(hasValue)
            {
                wait(); // wait if data not consumed
```

```
            }

            data = value;
            System.out.println("Produced: " + data);

            hasValue = true;
            notify(); // notify consumer
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    // Consume value
    synchronized void consume()
    {
        try
        {
            while(!hasValue)
            {
                wait(); // wait if no data
            }

            System.out.println("Consumed: " + data);

            hasValue = false;
            notify(); // notify producer
        }
        catch(Exception e)
        {
            System.out.println(e);session

        }
    }
}

// Producer Thread
class Producer extends Thread
{
    Shared s;

    Producer(Shared s)
```

```
    {
        this.s = s;
    }

    public void run()
    {
        for(int i = 1; i <= 5; i++)
        {
            s.produce(i);
        }
    }
}

// Consumer Thread
class Consumer extends Thread
{
    Shared s;

    Consumer(Shared s)
    {
        this.s = s;
    }

    public void run()
    {
        for(int i = 1; i <= 5; i++)
        {
            s.consume();
        }
    }
}

// Main Class
class Slip17Q2
{
    public static void main(String args[])
    {
        Shared s = new Shared();

        Producer p = new Producer(s);
        Consumer c = new Consumer(s);

        p.start();
```

```
        c.start();
    }
}
```

SLIP 18

Q1. Write a JSP program which accepts UserName in a TextBox. Display greet message to the user as “Welcome username”. [8 Marks]

```
index.html

<html>
<body>

<h2>Enter Username</h2>

<form action="welcome.jsp">
    Username:
    <input type="text" name="uname">
    <br><br>
    <input type="submit" value="Submit">
</form>

</body>
</html>

welcome.jsp

<html>
<body>

<%
    String name = request.getParameter("uname");

    if(name != null && !name.isEmpty())
    {
%>
        <h2>Welcome <%= name %></h2>
<%
    }
    else
```

```
    {
%>
        <h2>Please enter username</h2>
<%
    }
%>

</body>
</html>
```

Q2. Create a table as Course (Code, name, department_name, number_of_students). Insert values in the table. Write a JDBC program to perform following operations: i. Update number_of_students of “BSc Computer Science” to 120. ii. Delete the course details if number_of_students<10 [14 Marks]

```
CREATE TABLE Course
(
    Code INT,
    name VARCHAR(30),
    department_name VARCHAR(30),
    number_of_students INT
);

INSERT INTO Course VALUES (1,'BSc Computer Science','Science',50);
INSERT INTO Course VALUES (2,'BCA Science','Science',5);
INSERT INTO Course VALUES (3,'BBA','Commerce',200);

import java.sql.*;
import java.util.Scanner;

class CourseDB
{
    Connection con;
    Scanner sc = new Scanner(System.in);

    CourseDB()
    {
        try
        {
            Class.forName("org.postgresql.Driver");
```

```
        con = DriverManager.getConnection(
            "jdbc:postgresql://localhost:5432/postgres",
            "postgres",
            "password"
        );

        System.out.println("Connected to Database");
    }
    catch(Exception e)
    {
        System.out.println(e);
    }
}

// Insert (optional but good)
void insertCourse()
{
    try
    {
        System.out.print("Enter code: ");
        int code = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter name: ");
        String name = sc.nextLine();

        System.out.print("Enter department: ");
        String dept = sc.nextLine();

        System.out.print("Enter number of students: ");
        int num = sc.nextInt();

        PreparedStatement ps = con.prepareStatement(
            "INSERT INTO Course VALUES (?,?,?,?)"
        );

        ps.setInt(1, code);
        ps.setString(2, name);
        ps.setString(3, dept);
        ps.setInt(4, num);

        ps.executeUpdate();
```

```
            System.out.println("Inserted successfully");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    // i. Update
    void updateCourse()
    {
        try
        {
            PreparedStatement ps = con.prepareStatement(
                "UPDATE Course SET number_of_students=? WHERE name=?"
            );

            ps.setInt(1, 120);
            ps.setString(2, "BSc Computer Science");

            int rows = ps.executeUpdate();

            if(rows > 0)
                System.out.println("Updated successfully");
            else
                System.out.println("Course not found");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    // ii. Delete
    void deleteCourse()
    {
        try
        {
            PreparedStatement ps = con.prepareStatement(
                "DELETE FROM Course WHERE number_of_students < ?"
            );

            ps.setInt(1, 10);
```

```
            int rows = ps.executeUpdate();
            System.out.println(rows + " record(s) deleted");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    // Display (extra for marks)
    void display()
    {
        try
        {
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM Course");

            System.out.println("\n--- Course Table ---");
            while(rs.next())
            {
                System.out.println(
                    rs.getInt(1) + " | " +
                    rs.getString(2) + " | " +
                    rs.getString(3) + " | " +
                    rs.getInt(4)
                );
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

class Slip18Q2
{
    public static void main(String args[])
    {
        CourseDB obj = new CourseDB();
        Scanner sc = new Scanner(System.in);
```

```
        int choice;

        do
        {
            System.out.println("\n1. Insert");
            System.out.println("2. Update (BSc CS → 120)");
            System.out.println("3. Delete (<10 students)");
            System.out.println("4. Display");
            System.out.println("5. Exit");

            System.out.print("Enter choice: ");
            choice = sc.nextInt();

            switch(choice)
            {
                case 1: obj.insertCourse(); break;
                case 2: obj.updateCourse(); break;
                case 3: obj.deleteCourse(); break;
                case 4: obj.display(); break;
                case 5: System.out.println("Exit"); break;
                default: System.out.println("Invalid choice");
            }

        } while(choice != 5);
    }
}
```

SLIP 19

Q1. Write a java program to accept a String from a user and display each vowel from a String after every 3 seconds. [8 Marks]

```
import java.util.Scanner;

class VowelThread extends Thread
{
    String str;

    VowelThread(String str)
    {
        this.str = str;
    }
```

```java
    public void run()
    {
        try
        {
            for(int i = 0; i < str.length(); i++)
            {
                char ch = str.charAt(i);

                // Check vowel
                if(ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u'||
                   ch=='A'||ch=='E'||ch=='I'||ch=='O'||ch=='U')
                {
                    System.out.println("Vowel: " + ch);
                    Thread.sleep(3000); // 3 seconds
                }
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

class Slip19Q1
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        VowelThread t = new VowelThread(str);
        t.start();

        sc.close();
    }
}
```

Q2. Create table as Person(pno, pname, pcity, gender, bornyear). Write a JDBC program to perform search operation on Person table. i. Search all the person born in the year 1986. ii. Search all the females born between 2000-2005. [14 Marks]

```
CREATE TABLE Person
(
    pno INT,
    pname VARCHAR(30),
    pcity VARCHAR(30),
    gender VARCHAR(10),
    bornyear INT
);

INSERT INTO Person VALUES (1,'Asha','Pune','Female',1986);
INSERT INTO Person VALUES (2,'Rahul','Mumbai','Male',1986);
INSERT INTO Person VALUES (3,'Neha','Nashik','Female',2002);
INSERT INTO Person VALUES (4,'Sneha','Pune','Female',2004);
INSERT INTO Person VALUES (5,'Amit','Delhi','Male',2001);

import java.sql.*;
import java.util.Scanner;

class PersonDB
{
    Connection con;

    PersonDB()
    {
        try
        {
            Class.forName("org.postgresql.Driver");

            con = DriverManager.getConnection(
                "jdbc:postgresql://localhost:5432/postgres",
                "postgres",
                "password"
            );

            System.out.println("Connected to DB");
        }
        catch(Exception e)
        {
```

```
            System.out.println(e);
        }
    }

    // i. Born in 1986
    void search1986()
    {
        try
        {
            PreparedStatement ps = con.prepareStatement(
                "SELECT * FROM Person WHERE bornyear = ?"
            );

            ps.setInt(1, 1986);

            ResultSet rs = ps.executeQuery();

            System.out.println("\n--- Persons born in 1986 ---");
            while(rs.next())
            {
                System.out.println(
                    rs.getInt(1) + " | " +
                    rs.getString(2) + " | " +
                    rs.getString(3) + " | " +
                    rs.getString(4) + " | " +
                    rs.getInt(5)
                );
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    // ii. Females between 2000-2005
    void searchFemales()
    {
        try
        {
            PreparedStatement ps = con.prepareStatement(
                "SELECT * FROM Person WHERE gender=? AND bornyear
BETWEEN ? AND ?"
```

```
            );

            ps.setString(1, "Female");
            ps.setInt(2, 2000);
            ps.setInt(3, 2005);

            ResultSet rs = ps.executeQuery();

            System.out.println("\n--- Females born between 2000-2005
---");
            while(rs.next())
            {
                System.out.println(
                    rs.getInt(1) + " | " +
                    rs.getString(2) + " | " +
                    rs.getString(3) + " | " +
                    rs.getString(4) + " | " +
                    rs.getInt(5)
                );
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

class Slip19Q2
{
    public static void main(String args[])
    {
        PersonDB obj = new PersonDB();
        Scanner sc = new Scanner(System.in);

        int choice;

        do
        {
            System.out.println("\n1. Search persons born in 1986");
            System.out.println("2. Search females (2000-2005)");
            System.out.println("3. Exit");
```

```
            System.out.print("Enter choice: ");
            choice = sc.nextInt();

            switch(choice)
            {
                case 1: obj.search1986(); break;
                case 2: obj.searchFemales(); break;
                case 3: System.out.println("Exit"); break;
                default: System.out.println("Invalid choice");
            }

        } while(choice != 3);
    }
}
```

SLIP 20

Q1. Write a java program to accept 'N' student names through command line, store them into the appropriate Collection and display them by using Iterator. [8 Marks]

```
SLIP:20 Que:1.
import java.util.ArrayList;
import java.util.Iterator;

public class StudentNames {
    public static void main(String args[]) {

        ArrayList<String> list = new ArrayList<String>();

        if (args.length == 0) {
            System.out.println("Please enter student names as command
line arguments");
        } else {

            // Store names in ArrayList
            for (int i = 0; i < args.length; i++) {
                list.add(args[i]);
            }

            // Display using Iterator
```

```
            System.out.println("--- Student Names ---");

            Iterator<String> itr = list.iterator();

            while (itr.hasNext()) {
                System.out.println(itr.next());
            }
        }
    }
}
```

Q2. Create a JSP page, which accepts user name in a text box and greets the user according to the time on server side.
Example:
If user name is Admin Output: If it is morning then display message in red color as, Good morning Admin
Today's date: dd/mm/yyyy format
Current time: hh:mm:ss format If it is afternoon then display message in green color as, Good afternoon
Admin Today's date: dd/mm/yyyy format
Current time: hh:mm:ss format If it is evening then display message in blue color as, Good evening Admin
Today's date: dd/mm/yyyy format Current time: hh:mm:ss format
[14 Marks]

```
greet.jsp
<%@ page import="java.util.*" %>
<%@ page contentType="text/html" %>
<%
    String name = request.getParameter("uname");
    Calendar cal = new GregorianCalendar();
    int hour = cal.get(Calendar.HOUR_OF_DAY);
    int day = cal.get(Calendar.DAY_OF_MONTH);
    int month = cal.get(Calendar.MONTH) + 1;
    int year = cal.get(Calendar.YEAR);
    int minute = cal.get(Calendar.MINUTE);
    int second = cal.get(Calendar.SECOND);
    String message = "";
    String color = "";

    /* Time based greeting */
```

```
    if(hour < 12)
    {
        message = "Good morning " + name; color = "red";
    }
    else if(hour < 16)
    {
        message = "Good afternoon " + name; color = "green";
    }
    else
    {
        message = "Good evening " + name; color = "blue";
    }
%>

<h2 style="color:<%= color %>"> <%= message %> </h2>
    Today's date: <%= day %>/<%= month %>/<%= year %> <br>
    Current time: <%= hour %>:<%= minute %>:<%= second %>
```

SLIP 21

Q1. Write a java program to accept names of 'n' cities, insert same into array list collection and display the contents of same ArrayList. Display cities using Iterator interface. [8 Marks]

```
import java.util.*;

class Slip21Q1
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of cities: ");
        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        // Create ArrayList
        ArrayList<String> cities = new ArrayList<String>();

        // Accept city names
        for(int i = 1; i <= n; i++)
```

```
        {
            System.out.print("Enter city " + i + ": ");
            String city = sc.nextLine();
            cities.add(city);
        }

        // Display using Iterator
        System.out.println("\n--- City Names ---");

        Iterator<String> itr = cities.iterator();

        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }

        sc.close();
    }
}
```

Q2. Write a Servlet program to display number of times user has visited the page. (Use cookies) [14 Marks]

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class VisitServlet extends HttpServlet
{
    public void doGet(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException
    {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        int count = 0;

        // Get cookies
        Cookie cookies[] = req.getCookies();

        if(cookies != null)
```

```
        {
            for(int i = 0; i < cookies.length; i++)
            {
                if(cookies[i].getName().equals("visit"))
                {
                    count = Integer.parseInt(cookies[i].getValue());
                }
            }
        }

        // Increment count
        count++;

        // Create/Update cookie
        Cookie c = new Cookie("visit", String.valueOf(count));
        res.addCookie(c);

        // Display message
        if(count == 1)
        {
            out.println("<h2>Welcome! This is your first visit.</h2>");
        }
        else
        {
            out.println("<h2>You have visited this page " + count + "
times.</h2>");
        }
    }
}

<servlet>
    <servlet-name>VisitServlet</servlet-name>
    <servlet-class>VisitServlet</servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name>VisitServlet</servlet-name>
    <url-pattern>/VisitServlet</url-pattern>
</servlet-mapping>
```

SLIP 22

Q1. Write a JSP program to accept Name and Age of Voter and check whether he is eligible for voting or not. [8 Marks]

```
voter.html

<!DOCTYPE html>
<html>
<body>

<h2>Voter Eligibility Form</h2>

<form action="voter.jsp" method="post">
    Name: <input type="text" name="name"><br><br>
    Age: <input type="text" name="age"><br><br>

    <input type="submit" value="Check">
</form>

</body>
</html>


voter.jsp

<%@ page import="java.io.*" %>
<%@ page contentType="text/html" %>

<%
String name = request.getParameter("name");
int age = Integer.parseInt(request.getParameter("age"));

if(age >= 18)
{
%>
    <h2 style="color:green;">
        <%= name %> is eligible for voting.
```

```
    </h2>
<%
}
else
{
%>
    <h2 style="color:red;">
        <%= name %> is NOT eligible for voting.
    </h2>
<%
}
%>
```

Q2. Consider a table as District (Name, area, population). Write a JDBC program to implement search and delete operations on a District table. [14 Marks]

```
import java.sql.*;
import java.util.Scanner;

class DistrictDB
{
    Connection con;
    Scanner sc = new Scanner(System.in);

    DistrictDB()
    {
        try
        {
            Class.forName("org.postgresql.Driver");

            con = DriverManager.getConnection(
                "jdbc:postgresql://localhost:5432/postgres",
                "postgres",
                "password"
            );

            System.out.println("Connected to Database");
        }
        catch(Exception e)
        {
```

```
            System.out.println(e);
        }
    }

    // Insert (extra for marks)
    void insertDistrict()
    {
        try
        {
            System.out.print("Enter District Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Area: ");
            double area = sc.nextDouble();

            System.out.print("Enter Population: ");
            int pop = sc.nextInt();
            sc.nextLine();

            PreparedStatement ps = con.prepareStatement(
                "INSERT INTO District VALUES (?,?,?)"
            );

            ps.setString(1, name);
            ps.setDouble(2, area);
            ps.setInt(3, pop);

            ps.executeUpdate();
            System.out.println("Inserted successfully");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    // i. Search
    void searchDistrict()
    {
        try
        {
            System.out.print("Enter District Name to search: ");
            String name = sc.nextLine();
```

```java
        PreparedStatement ps = con.prepareStatement(
            "SELECT * FROM District WHERE Name=?"
        );

        ps.setString(1, name);

        ResultSet rs = ps.executeQuery();

        if(rs.next())
        {
            System.out.println("Found:");
            System.out.println(
                rs.getString(1) + " | " +
                rs.getDouble(2) + " | " +
                rs.getInt(3)
            );
        }
        else
        {
            System.out.println("District not found");
        }
    }
    catch(Exception e)
    {
        System.out.println(e);
    }
}

// ii. Delete
void deleteDistrict()
{
    try
    {
        System.out.print("Enter District Name to delete: ");
        String name = sc.nextLine();

        PreparedStatement ps = con.prepareStatement(
            "DELETE FROM District WHERE Name=?"
        );

        ps.setString(1, name);
```

```
            int rows = ps.executeUpdate();

            if(rows > 0)
                System.out.println("Deleted successfully");
            else
                System.out.println("District not found");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }

    // Display (extra)
    void display()
    {
        try
        {
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM District");

            System.out.println("\n--- District Table ---");
            while(rs.next())
            {
                System.out.println(
                    rs.getString(1) + " | " +
                    rs.getDouble(2) + " | " +
                    rs.getInt(3)
                );
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

class Slip22Q2
{
    public static void main(String args[])
    {
        DistrictDB obj = new DistrictDB();
```

```java
        Scanner sc = new Scanner(System.in);

        int choice;

        do
        {
            System.out.println("\n1. Insert");
            System.out.println("2. Search");
            System.out.println("3. Delete");
            System.out.println("4. Display");
            System.out.println("5. Exit");

            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch(choice)
            {
                case 1: obj.insertDistrict(); break;
                case 2: obj.searchDistrict(); break;
                case 3: obj.deleteDistrict(); break;
                case 4: obj.display(); break;
                case 5: System.out.println("Exit"); break;
                default: System.out.println("Invalid choice");
            }

        } while(choice != 5);
    }
}
```

SLIP 23

Q1. Write a servlet program to accept username and display message as "Have a Nice day!!!" with username. [8 Marks]

```
username.html

<!DOCTYPE html>
<html>
<body>

<h2>Enter Username</h2>
```

```
<form action="HelloServlet" method="post">
    Username: <input type="text" name="uname"><br><br>
    <input type="submit" value="Submit">
</form>

</body>
</html>


HelloServlet.java

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet
{
    public void doPost(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException
    {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        String name = req.getParameter("uname");

        out.println("<h2>Have a Nice Day!!! " + name + "</h2>");
    }
}

<servlet>
    <servlet-name>HelloServlet</servlet-name>
    <servlet-class>HelloServlet</servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name>HelloServlet</servlet-name>
    <url-pattern>/HelloServlet</url-pattern>
</servlet-mapping>
```

Q2. Create table Emp(ENo EName Salary). Write a JDBC program to delete the details of employee on the basis of employee number. Accept employee number through command line. Display all the details of employees. [14 Marks]

```
import java.sql.*;

public class EmpJDBC {
    public static void main(String[] args) {

        // Check command line argument
        if (args.length == 0) {
            System.out.println("Please provide Employee Number as a
command line argument.");
            return;
        }

        Connection con = null;
        PreparedStatement ps1 = null;
        PreparedStatement ps2 = null;
        ResultSet rs = null;

        try {
            // 1. Load Driver
            Class.forName("org.postgresql.Driver");

            // 2. Establish Connection
            con = DriverManager.getConnection(
                "jdbc:postgresql://192.168.5.200:5432/bcaty13",
                "bcaty13", ""
            );

            System.out.println("Connection Successful");

            // 3. Get employee number from command line
            int eno = Integer.parseInt(args[0]);

            // 4. Delete employee record
            ps1 = con.prepareStatement("DELETE FROM Emp WHERE ENo =
?");
            ps1.setInt(1, eno);

            int rowsDeleted = ps1.executeUpdate();
```

```
            // Added a friendly check to give the user feedback
            if (rowsDeleted > 0) {
                System.out.println("Employee " + eno + " deleted
successfully.");
            } else {
                System.out.println("Employee " + eno + " not found in
the database.");
            }

            // 5. Display all remaining employees
            ps2 = con.prepareStatement("SELECT * FROM Emp");
            rs = ps2.executeQuery();

            System.out.println("\nRemaining Employees:");
            System.out.println("ENo\tEName\tSalary");
            System.out.println("------------------------");

            while (rs.next()) {
                int id = rs.getInt("ENo");
                String name = rs.getString("EName");
                double salary = rs.getDouble("Salary");

                System.out.println(id + "\t" + name + "\t" + salary);
            }

        } catch (Exception e) {
            // FIXED: Replaced Sop(e) with standard print statement
            System.out.println("Database Error: " + e.getMessage());
        }
        }
    }
```

SLIP 24

Q1. Write program to create a link list of integer objects. Perform following operations on link list. i. Add element at first position ii. Display the size of link list [8 Marks]

```java
import java.util.*;


class Slip24Q1
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);


        LinkedList<Integer> list = new LinkedList<Integer>();


        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();


        // Add elements at FIRST position
        for(int i = 1; i <= n; i++)
        {
            System.out.print("Enter number " + i + ": ");
            int num = sc.nextInt();


            list.add(num); // important
        }

        System.out.print("Enter element to add at first position: ");
        int first = sc.nextInt();
        list.addFirst(first);


        // Display using Iterator
        System.out.println("\n--- Linked List Elements ---");


        Iterator<Integer> itr = list.iterator();


        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }
```

```
        // Display size
        System.out.println("\nSize of Linked List: " + list.size());


        sc.close();
    }
}
```

Q2. Write a Servlet program to display number of times user has visited the page. (Use Session) [14 Marks]

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SessionServlet extends HttpServlet
{
    public void doGet(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException
    {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        // Get session object
        HttpSession session = req.getSession();

        Integer count = (Integer) session.getAttribute("visitCount");

        if(count == null)
        {
            count = 1;
            out.println("<h2>Welcome! This is your first visit.</h2>");
        }
        else
        {
            count++;
            out.println("<h2>You have visited this page " + count + "
times.</h2>");
        }
```

```
        // Set updated count in session
        session.setAttribute("visitCount", count);
    }
}
```

SLIP 25

Q1. Write JDBC program to check database connection to Emp table (Eno, ename, salary). Display appropriate message. [8 Marks]

```
import java.sql.*;

class Slip25Q1
{
    public static void main(String args[])
    {
        try
        {
            // 1. Load Driver
            Class.forName("org.postgresql.Driver");

            // 2. Establish Connection
            Connection con = DriverManager.getConnection(
                "jdbc:postgresql://localhost:5432/postgres",
                "postgres",
                "password"
            );

            // 3. Check connection
            if(con != null)
            {
                System.out.println("Connected to Emp Database
successfully!");

                // Optional: Check Emp table
                Statement st = con.createStatement();
```

```
                ResultSet rs = st.executeQuery("SELECT * FROM Emp");

                System.out.println("Emp Table exists and accessible.");
            }
            else
            {
                System.out.println("Connection failed!");
            }

            con.close();
        }
        catch(Exception e)
        {
            System.out.println("Error: " + e);
        }
    }
}
```

Q2. Write a servlet program to display IP address, browser type, current date and time of server. [14 Marks]

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.Date;

public class InfoServlet extends HttpServlet
{
    public void doGet(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException
    {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        // IP Address
        String ip = req.getRemoteAddr();

        // Browser Type
        String browser = req.getHeader("User-Agent");

        // Current Date & Time
```

```
        Date d = new Date();

        out.println("<h2>Client Information</h2>");

        out.println("IP Address: " + ip + "<br><br>");
        out.println("Browser: " + browser + "<br><br>");
        out.println("Current Date & Time: " + d);
    }
}
```